

Topic

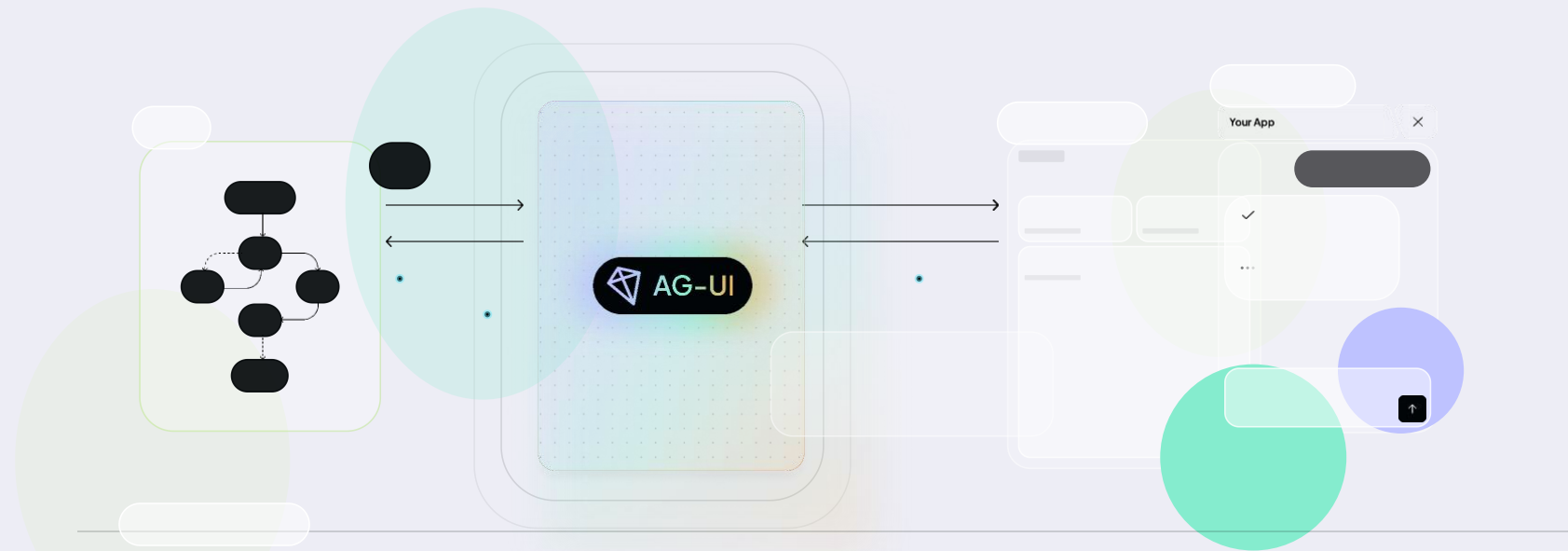
Building Agentic Apps with CopilotKit and AG-UI



Ran Shemtov

Founding Engineer, CopilotKit







Weekly Installs

● AG-UI ● CopilotKit

Growing 24% MoM

Total: 120K+



CopilotKit 

CopilotKit 

**1 million agent-user interactions
a week powered by CopilotKit**





**SaaS
Copilots**



**Productivity
Copilots**



Search HubSpot

 K















Assistant

CopilotKit

Bookmarks

CRM

Marketing

Content

Sales

Commerce

Service

Data Management

Automation

Reporting

Add assistant



Ask me anything.

 Add apps  

IN PROGRESS 3

- Enterprise Order Managem...
Generating
- Analyze Tab vs Agent Usag...
Generating
- Fix PR Comments Fetching I...
Generating

READY FOR REVIEW 3

- PyTorch MNIST Experim... 1m
+162 -37 · Done, configurabl...
- Set up Cursor Rules fo... 31m
+37 -0 · Set up Cursor Rules f...
- Bioinformatics Tools 46m
+135 -21 · Bioinformatics Tools

PyTorch MNIST Experiments

Add mixed precision training, learning rate scheduling, and proper validation. Also create an experiment config system so I can easily run different hyperparameter settings.

proper configuration management. Let me start by rewriting the training module:

train_model.py +156 -34

Now let me update the evaluation module to save results and generate a detailed report:

evaluation.py +24 -6

Let me create a command-line runner script to make experiments easy to execute:

run_experiment.py +29

Done, configurable MNIST experiment framework with AMP and reports.

Cursor

train_model.py X

run_experiment.py

config.yaml

```
import torch
import torch.nn as nn

- from torch.utils.data import DataLoader
+ from torch.utils.data import DataLoader, random_split
- from torchvision import datasets
+ from torchvision import datasets, transforms
+ from tqdm import tqdm
+ import yaml
+ from pathlib import Path
+ import json

- def get_dataloaders(batch_size=64):
-     transform = transforms.Compose([transforms.ToTensor()])
-     train = datasets.MNIST(root="data", train=True, download=True)
-     test = datasets.MNIST(root="data", train=False, download=True)
-     return DataLoader(train, batch_size=batch_size, shuffle=True)

+ def load_config(config_path="experiments/config.yaml"):
+     with open(config_path) as f:
+         return yaml.safe_load(f)
```


The past 30 years of the internet

Request



Response

Agentic Software



Long running (require streaming)



Structured + Unstructured I/O (text & voice + tool calls & state updates...)



Require composition (hand-off control to subagents)



Nondeterministic UI (inversion of control)

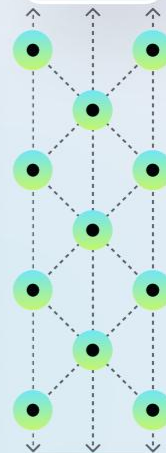
And more



Frontend execution handoff

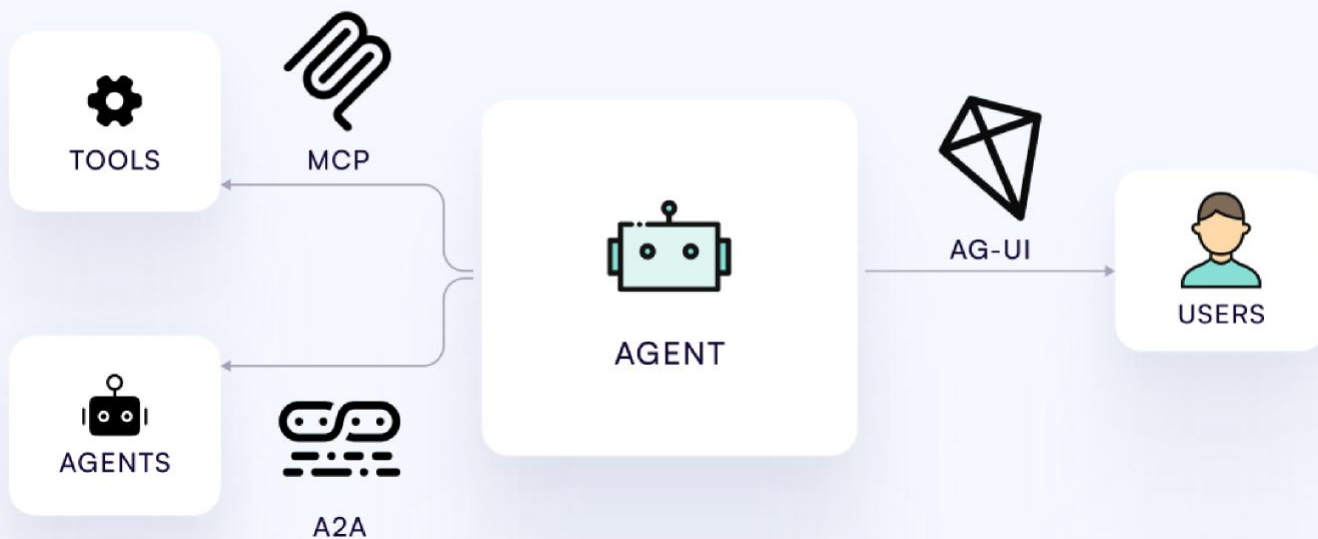


AGENTS

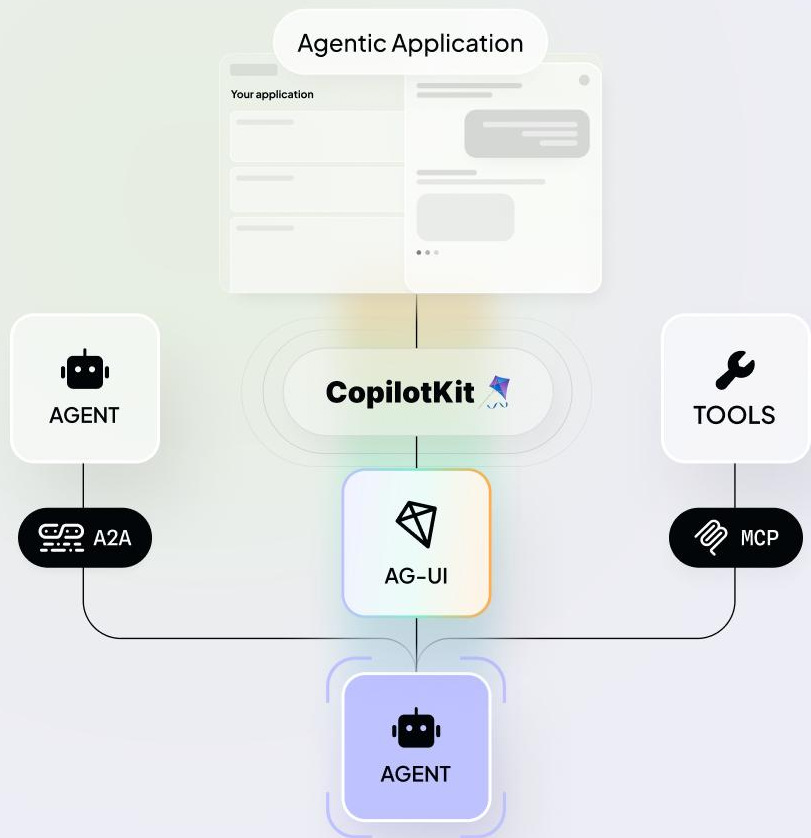


USER

The Agent Protocol Stack



CopilotKit & AG-UI Connect Agents and Applications



AG-UI

Event-based protocol connecting agentic backends & agentic frontends.

CopilotKit

SDKs & Self-Hostable Platform for building agentic applications - purpose built for the agentic paradigm.



LangChain
487,364 followers
3w ·

AI-Powered Canvas Template

Brandon from CopilotKit released a production template for AI canvas apps. Built with LangGraph for agent coordination, it delivers real-time UI-AI synchronization through a Python-Next.js stack.

Watch the walkthrough: <https://finkd.in/gkDVTwQH>

Build an Open AG-UI Canvas with LangGraph



CopilotKit

You and 375 others · 6 comments · 35 reposts

Developers · Products · Solutions · Events · Learn · Community · Developer Program · Blog

Search all articles...

Delight users by combining ADK Agents with Fancy Frontends using AG-UI

SEPT. 26, 2023
Alan Bounie
Senior Technical Product Manager

Delight users by combining ADK Agents with Fancy Frontends using AG-UI

For developers building generative AI applications, the last year has been a whirlwind of progress. We've seen incredible advancements in backend agents systems that can reason, plan, and execute complex tasks. But a powerful agent is only half the story. How do you seamlessly connect that intelligence to your application's frontend? How do you create a truly collaborative experience between your user and your AI?

Today, we're excited to highlight a new integration that directly addresses this challenge: the **Agent Development Kit (ADK)** now works with **AG-UI**, an open protocol for building rich, interactive AI user experiences. This combination empowers you to build sophisticated AI agents on the backend and give them a production-ready, collaborative frontend with minimal effort.

NET Conf 2025

Microsoft

AG-UI Overview

The Agent User Interaction (AG-UI) Protocol

AG-UI is an open protocol that enables you to build web-based AI agent applications with advanced features like real-time streaming, state management, and interactive UI components. The Agent Framework AG-UI integration provides seamless connectivity between your agents and web clients.

NET Conf 2025 - Day 1

dotnet 541K subscribers

Subscribe

792 · Share · Clip · Save

Building agentic copilots with CopilotKit and Mastra

Sam Bhagwat · Sep 18, 2025

Mastra is an agentic backend: it provides you the primitives like agents and workflows, to create specialized agents that can react.

But every backend needs a great agentic frontend.

One of the frontends we recommend is **CopilotKit**. CopilotKit provides interactive React components that hook into the Mastra streaming protocol and let you customize the user experience.



Agent Development Kit

Build chat experiences with AG-UI and CopilotKit

As an agent builder, you want users to interact with your agents through a rich and responsive interface. Building UIs from scratch requires a lot of effort, especially to support streaming even client state. That's exactly what **AG-UI** was designed for - rich user experiences directly connect an agent.

AG-UI provides a consistent interface to empower rich clients across technology stacks, from the web and even the command line. There are a number of different clients that support AG-UI:

- CopilotKit provides tooling and components to tightly integrate your agent with web applications
- Clients for Kotlin, Java, Go, and CLI implementations in TypeScript

This tutorial uses CopilotKit to create a sample app backed by an ADK agent that demonstrates of the features supported by AG-UI.

Microsoft Ignite
November 17-21, 2025

Learn · Documentation · Training · Q&A · Tools

Agent Framework Overview · Quick Start · Tutorials · User guides

Find by title

Agent Framework
Quick Start Guide

Tutorials

User Guide

Integrations

AG-UI

Overview

Getting Started

Backend Tool Rendering

Frontend Tool Rendering

Security Considerations

Support

Migration Guide

API Reference Guide

NET API reference

Learn / Microsoft Agent Framework /

Ask Learn · Focus mode

AG-UI Integration with Agent Framework

Choose a programming language

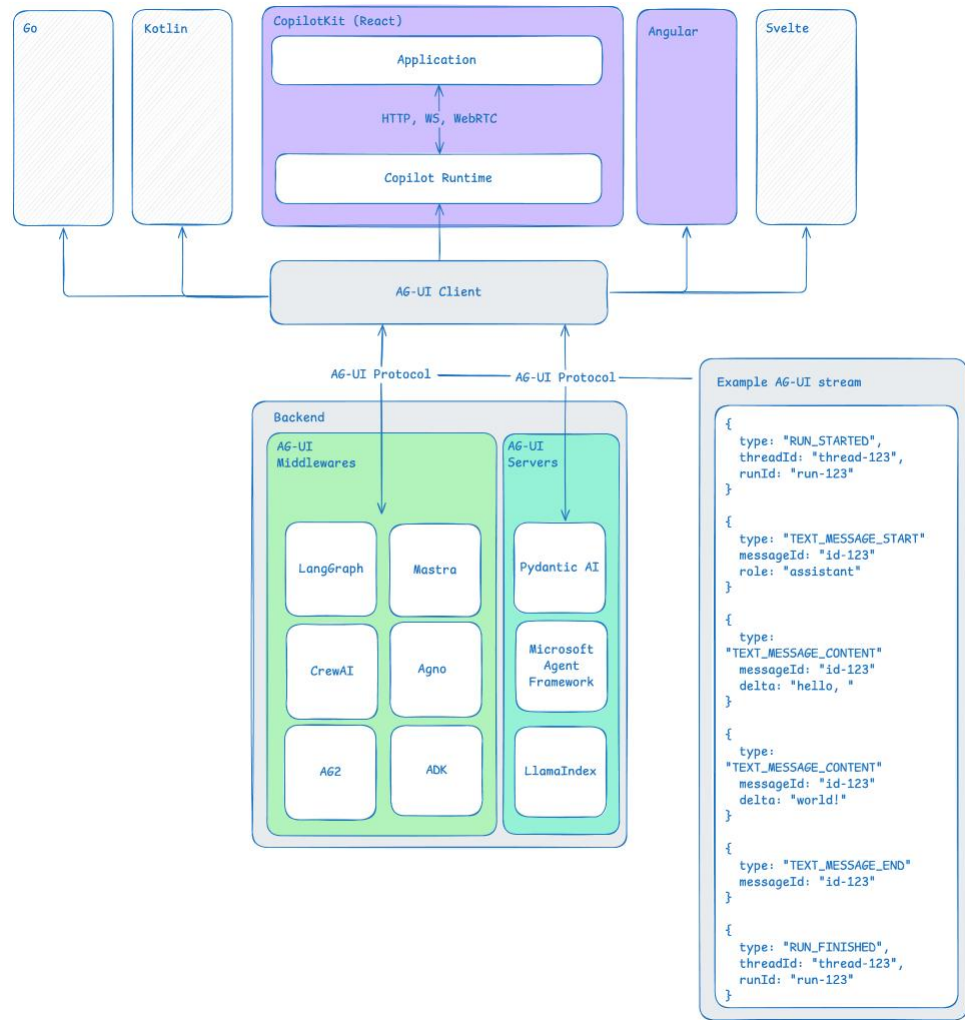
Cx Python

AG-UI is a protocol that enables you to build web-based AI agent applications with advanced features like real-time streaming, state management, and interactive UI components. The Agent Framework AG-UI integration provides seamless connectivity between your agents and web clients.

What is AG-UI?

AG-UI is a standardized protocol for building AI agent interfaces that provides:

- Remote Agent Hosting:** Deploy AI agents as web services accessible by multiple clients



•

•

•

-
-
-

WEATHER IN

New York City



38 °F

Updated 11:02 AM

The current temperature in New York City is 38 degrees.

```
1  # Agent
2
3  def get_weather(location: str):
4      return {temp: 38, humidity: 50}
5
6  agent = agent(
7      "gpt-5.1",
8      tools=[get_weather]
9  )
10
11  agent.ag_ui.serve()
```



```
1  // Frontend
2
3  import {
4      CopilotChat
5      useRenderTool
6  } from "@copilotkit/react-ui/v2"
7
8  useRenderTool({
9      name: "get_weather",
10     render: ({ args, result }) => {
11         return <WeatherCard
12             location={args.location}
13             temp={result.temp}
14         />;
15     },
16 });
17
18 return <CopilotChat/>
```

What's the weather in New York City?

```
1 // Frontend
2
3 import {
4   CopilotChat
5   useRenderTool
6 } from "@copilotkit/react-ui/v2"
7
8 useRenderTool({
9   name: "get_weather",
10  render: ({ args, result }) => {
11    return <WeatherCard
12      location={args.location}
13      temp={result.temp}
14    />;
15  },
16 });
17
18 return <CopilotChat/>
```



The current temperature in New York City is 38 degrees.



+ Type a message...



AI can make mistakes. Please verify important information.

What's the weather in New York City?

```
1 // Frontend
2 const { agent } = useAgent()
3
4 return (
5   <div>
6     {agent.messages.map(
7       (message) => <div>
8         { message.content }
9         { message.ui() }
10      </div>
11    )}
12   </div>
13 )
```



The current temperature in New York City is 38 degrees.



+ Type a message...



AI can make mistakes. Please verify important information.

•

•

•

•

•

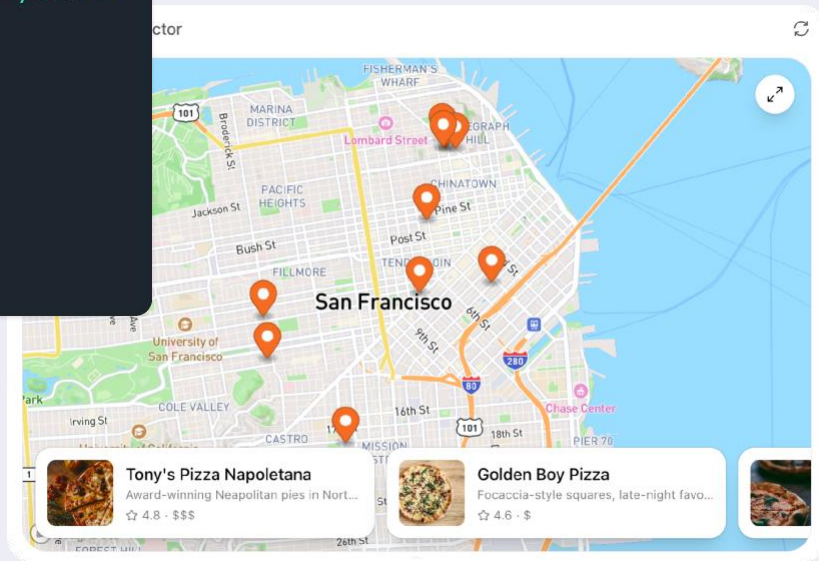
•

•

```

1  <!-- ChatGPT UI -->
2  <html>
3    <body>
4      <!-- ChatGPT wraps your app in sandboxed iframes -->
5      <iframe src="https://web-sandbox.oaiusercontent.com/...">
6        <iframe src="https://your-app.example.com/">
7          <!-- Your ChatGPT App SDK UI runs here -->
8        </iframe>
9      </iframe>
10    </body>
11  </html>
12

```



•

•

•

•

•

•

•

•

•

```
1 <Card>
2   <Image src={airline.logo} size={24} />
3
4   <Text value={` ${airline.name} ${number}`} />
5
6   <Text value={` ${departure.city} → ${arrival.city}`} />
7 </Card>
8
```



Pan America PA 845
San Francisco → London

•

•

•

•

•



•

•

•

```
1 // Agent
2 class StateSchema:
3     name: str
4
5 agent = create_agent(
6     "gpt-5.1",
7     tools=tools
8     state=StateSchema
9 )
10
11 agent.ag_ui.serve()
12
```



```
1 // Frontend
2 import {
3     useAgent
4 } from "@copilotkit/react-core/v2"
5
6 const { agent } = useAgent();
7
8 return (
9     <div>
10         { agent.state.name }
11     </div>
12 )
```

```
1 // Agent
2 class StateSchema:
3     counter: str
4
5 agent = create_agent(
6     "gpt-5.1",
7     tools=tools
8     state=StateSchema
9 )
10
11 agent.ag_ui.serve()
12
```

```
1 // Frontend
2 import {
3     useAgent
4 } from "@copilotkit/react-core/v2"
5
6 const { agent } = useAgent();
7
8 return (
9     <button
10         onClick={agent.setState({
11             agent.state.counter++
12         })}
13     >
14         Increase counter
15     </button>
16 )
```

```
1 // Agent
2 class StateSchema:
3     document: str
4
5 agent = create_agent(
6     "gpt-5.1",
7     tools=[generate_document]
8     state=StateSchema
9 )
10
11 agent.ag_ui.serve()
12
```

```
1 // Frontend
2 import {
3     useAgent
4 } from "@copilotkit/react-core/v2"
5
6 const { agent } = useAgent();
7 const [changes, setChanges] = ...
8
9 return (
10     <main>
11         <p>{agent.document}</p>
12         <button
13             onClick={agent.setState({
14                 document: changes
15             })}
16         >
17             Save changes
18         </button>
19     </main>
20 )
21
```

Todo Board

Manage your tasks with the help of your AI assistant! 🦾

Todo

+ Add Task

No tasks yet

In-Progress

+ Add Task



Build a todo app

Create an AI-powered todo application

Done

+ Add Task



Learn CopilotKit

Explore the amazing features of CopilotKit!

Todo Assistant

Help

Debug



🦾 Hi! I can help you manage your todos.



Add Todos

Change Theme

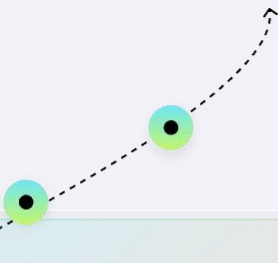
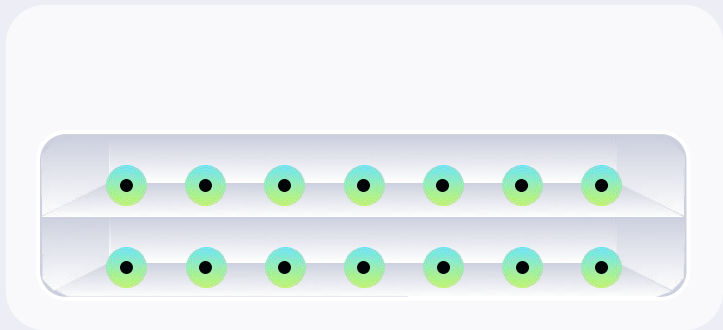
Update Status

Full Send

Manage Tasks

Read State

-
-
-





[README](#) [Contributing](#) [MIT license](#)

🔗 AG-UI: The Agent-User Interaction Protocol

AG-UI is an open, lightweight, event-based protocol that standardizes how AI agents connect to user-facing applications. Built for simplicity and flexibility, it enables seamless integration between AI agents, real time user context, and user interfaces.

[Version v0.0.41](#) [License MIT](#) [Discord 100 online](#)

[Join our Discord →](#) [Read the Docs →](#) [Go to the AG-UI Dojo →](#) [Follow us →](#)


YOUR APPLICATION


AG-UI PROTOCOL

 LangGraph

 LlamaIndex

 Agno

 AI SDK

 ASE

 crewAI

 Pydantic AI

 mastra

 Google ADK

COMING SOON

 Microsoft Agent Framework

 Amazon Bedrock

 STRANDS AGENTS

AND MORE